

## Model Practical

2/8

SET-10:-

- 1) write a C program to Hill cipher succumbs to a known plaintext attack if sufficient plain text - ciphertext pairs are provided. It is even easier to solve the Hill cipher if a chosen plaintext attack can be mounted.
- 2) write a C program for subkey generation in DES. The first subkey is derived from the resulting string by a left shift of one bit and condition by XORing constant that depend on block size.
- 3) write a C program for perform a letter frequency attack on monoalphabetic substitution cipher without human intervention.
- 4) write a C program for Diffie - Hellman



1)

## Hill cipher:

### AIM:

Analyze and execute the known plaintext value Hill cipher with known plaintext value and the cipher text pairs are provided.

### Algorithm:-

- => START
- => Read the  $2 \times 2$  matrix
- => Read the plain text.
- => Multiply the key matrix with plain text
- => Display cipher text
- => STOP.

### coding:

```
#include <stdio.h>
int main()
{
    int key[2][2], text[2], cipher[2], i, j;
    printf("Enter 2x2 Matrix:");
    for (i=0; i<2; i++)
    {
        for (j=0; j<2; j++)
        {
            scanf("%d", &key[i][j]);
        }
    }
}
```

the output was verified.



```
printf ("Enter the two plaintext value:");  
scanf ("%d %d", &text[0], &text[1]);  
for (i=0; i<2; i++)  
{  
    cipher[i] = 0;  
}  
for (j=0; j<2; j++)  
{  
    cipher[i] += key[i][j] * text[j];  
}  
cipher[i] %= 26;  
for (i=0; i<2; i++)  
{  
    printf ("cipher text", cipher[i]);  
}  
return 0;  
}
```

Input: Enter 2x2 matrix:  $\begin{bmatrix} 3 & 3 \\ 2 & 5 \end{bmatrix}$

Enter plaintext value: 7, 8

Output:

cipher text : 19 2

result:

Thus the program has been executed successfully and the output was verified.



2)

## Subkey Generation in CMAC

AIM:

To generate the subkey in CMAC. First subkey is derived from the resulting string by left shift of one bit.

Algorithm:-

- => START
- => Read L value
- => ~~Read~~ Left shift L can be key generation of  $k_1$ .
- => The most significant bit  $k_1$
- => Left shift  $k_1$  can be using key generation of  $k_2$ .
- => The most significant is  $k_2$ .
- => Display  $k_1$  and  $k_2$ .
- => STOP.

Coding:

```
#include <stdio.h>
int main()
{
    signed long long L, k1, k2;
    signed long long Rb: 0x1B;
    printf("Enter L value:")
    scanf("%llx", &L);
```



```
if (L = k1 < 1)
{
    Rb = (L < (1ULL < 63));
}
if (k1 = k2 < 1)
{
    Rb = (k1 < (1ULL < 63));
}
printf ("%11x", k1);
printf ("%11x", "The k2 is:", k2);
return 0;
}
```

Input:

Enter L value : 2

Output:

The k1 is : 4

The k2 is : 8

Result:

Thus, the program has been executed successfully and the output was verified.



3)

## Letter frequency attack.

AIM:-

To perform a letter frequency attack on monoalphabetic substitution cipher.

Algorithm:-

=> START

=> Read the cipher text.

=> Count the frequency of each alphabet letter.

=> Arrange the letters According to their frequency.

=> Generate possible plaintext

=> Display most likely plaintext.

=> STOP.

coding:-

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <ctype.h>
```

```
int main()
```

```
{
```

```
    char text[500];
```

```
    int freq[26] = {0};
```

```
    int i, j, temp;
```

```
    char letters[26];
```

```
    printf("Enter cipher text:");
```



```
fgets (text, size of text);
```

```
for (i=0; text[i] != '\0'; i++);
```

```
{
```

```
    if (isalpha(text[i]))
```

```
    {
```

```
        text[i] = (toupper(text[i]));
```

```
    }
```

```
}
```

```
for (i=0; i < 26; i++)
```

```
    letters[i] = 'A' + i;
```

```
for (i=0; i < 26; i++)
```

```
{
```

```
    for (j=i+1; j < 26; j++)
```

```
    { if (freq[i] < freq[j])
```

```
        { temp = freq[i];
```

```
          freq[i] = freq[j];
```

```
          freq[j] = temp;
```

```
          letters[i] = letters[j];
```

```
          letters[j] = temp;
```

```
        }
```

```
    }
```

```
}
```

```
printf("\n letter frequency");
```

```
for (i=0; i < 26; i++)
```

```
{ if (freq[i] > 0)
```

```
    printf("%c : %d", letters[i], freq[i]);
```

```
}
```

```
return 0;
```



Input:

Enter cipher text: K H O O R Z R U O G

Output:

Letter frequency analysis:

O : 3

R : 2

K : 1

H : 1

Z : 1

U : 1

G : 1

Result:-

Thus, the program has been executed successfully and the output was verified.



## Diffie- Hellman key Exchange:-

AIM:-

To perform a Diffie- Hellman key Exchange.

Algorithm:-

- ⇒ START
- ⇒ Read public value of  $q$  and  $a$ ;
- ⇒ Read Alice private key  $x$
- ⇒ Read Bob private key  $y$ .
- ⇒ Calculate Alice public key  $A = a^x \bmod q$
- ⇒ Calculate Bob public key  $B = a^y \bmod q$
- ⇒ Calculate shared secret key.
- ⇒ Display output
- ⇒ STOP.

Coding:-

```
#include <stdio.h>
```

```
long long power(long long a, long long b, long long mod)
```

```
{
```

```
    long long result = 1;
```

```
    while (b > 0)
```

```
    {
```

```
        result = (result * a) % mod;
```

```
        b--;
```

```
    }
```

```
    return result;
```

```
}
```

```
int main()
```

```
{
```



```

long long q, a, x, y;
long g, long A, B, key1, key2;
printf("Enter prime number q: ")
scanf("%lld", &q);
printf("Enter public number a: ");
scanf("%lld", &a);
printf("Alice secret key x: ");
scanf("%lld", &x);
printf("Bob secret key y: ");
scanf("%lld", &y);
A = power(a, x, q);
B = power(p, y, q);
key1 = power(B, x, q);
key2 = power(A, y, q);
printf("Alice shared key: %d", key1);
printf("Bob shared key: %d", key2);
return 0;
}

```

Input:

Enter prime number: 23  
 Enter public number: 5  
 Enter Alice secret key: 6  
 Enter Bob secret key: 15

Output:

Alice send: 8  
 Bob send: 19  
 Alice shared key: 2  
 Bob shared key: 2

Result:

Thus, the program has been executed successfully and the output was verified.